DATA STRUCTURES AND ALGORITHMS

Complete Chapter-Wise Notes

Definitions • Examples • Algorithms • Complexity • Exam Questions

English Edition

A study guide for Computer Science students

How to Use These Notes

Each chapter contains a clear definition, key concepts, examples, and exam-focused revision points. Complexity values are standard asymptotic bounds; actual performance depends on implementation and input.

Contents

- 1. Introduction to Data Structures
- 2. Algorithms and Complexity Analysis
- 3. Arrays
- 4. Strings
- 5. Linked Lists
- 6. Stacks
- 7. Queues and Circular Queues
- 8. Recursion
- 9. Searching Algorithms
- 10. Sorting Algorithms
- 11. Trees and Binary Trees
- 12. Binary Search Trees
- 13. Tree Traversals
- 14. Heaps and Priority Queues
- 15. Hashing and Hash Tables
- 16. Graphs
- 17. BFS and DFS
- 18. Greedy Algorithms
- 19. Divide and Conquer
- 20. Dynamic Programming
- 21. Backtracking
- 22. Time and Space Complexity
- 23. Advanced Topics: AVL, Trie, DSU and File Structures
- 24. Exam Questions and Quick Revision

Chapter 1: Introduction to Data Structures

Definition

A **data structure** is a way of organizing and storing data so that it can be accessed, updated, inserted, deleted, and processed efficiently.

Example: A student record system may store student IDs in an array, maintain a queue of waiting students, and use a tree to represent an organization hierarchy.

Types of Data Structures

Type	Definition	Examples
Primitive	Basic built-in data types.	Integer, character, Boolean, floating-point number
Linear	Elements are arranged in a sequence.	Array, linked list, stack, queue
Non-linear	Elements form hierarchical or network relationships.	Tree, graph
Static-size	Capacity is fixed after creation in the usual implementation.	Fixed-size array
Dynamic	Structure can grow or shrink during execution.	Linked list; dynamic array

Why Data Structures Matter

- They help manage large amounts of data.
- They can reduce searching, insertion, and deletion costs.
- They are used in databases, operating systems, networks, compilers, and applications.

Quick Check

Example: Choose a queue for customers served in arrival order; choose a stack for undo operations.

Chapter 2: Algorithms and Complexity Analysis

Definition

An **algorithm** is a finite sequence of clear, executable steps used to solve a problem or perform a computation.

Characteristics

- Input: accepts zero or more inputs.
- Output: produces at least one result.
- Definiteness: each step is unambiguous.
- Finiteness: it terminates after a finite number of steps.
- Effectiveness: its steps are practically executable.

Example: Add Two Numbers

```
START
INPUT A, B
SUM = A + B
PRINT SUM
STOP
```

If A = 12 and B = 8, the output is 20.

Best, Average, and Worst Cases

Best case is the most favorable input; **average case** describes expected performance under an input model; **worst case** is the maximum cost for inputs of a given size.

Asymptotic Notations

Notation	Meaning	Use
O(f(n))	Asymptotic upper bound	Often used to express an upper growth bound
Ω(f(n))	Asymptotic lower bound	Expresses a lower growth bound
Θ(f(n))	Tight asymptotic bound	Upper and lower bounds of the same order

Common Growth Rates

Complexity	Name	Example
O(1)	Constant	Array index access
O(log n)	Logarithmic	Binary search
O(n)	Linear	Scan a list
O(n log n)	Linearithmic	Merge sort
O(n²)	Quadratic	Two full nested loops
O(2■)	Exponential	Some exhaustive recursive searches

Chapter 3: Arrays

Definition

An **array** is a linear data structure that stores a sequence of elements, typically of the same type, in contiguous memory locations. Elements are accessed by index.

Example: A = [10, 20, 30, 40]. With zero-based indexing, A[2] is 30.

Types

- One-dimensional array: a single sequence.
- Two-dimensional array: rows and columns, such as a matrix.
- Multidimensional array: three or more dimensions.

Operations and Complexity

Operation	Typical time	Explanation
Access by index	O(1)	Read a known position
Linear search	O(n)	Check elements one by one
Binary search	O(log n)	Works on sorted arrays
Insert/delete at beginning or middle	O(n)	Elements may need shifting
Traverse	O(n)	Visit all elements

Example: Find the Largest Element

```
largest = A[0]
FOR each element x in A
    IF x > largest
        largest = x
RETURN largest
```

Example: For A = [4, 9, 2, 7], the largest element is 9. Time is O(n), extra space is O(1).

Advantages and Limitations

- Advantages: fast indexed access and simple traversal.
- Limitations: fixed-size arrays cannot grow automatically; middle insertions and deletions may require shifting.

Chapter 4: Strings

Definition

A **string** is a sequence of characters used to represent text.

Example: The string "HELLO" contains five characters.

Common Operations

- Length: count characters.
- Concatenation: join strings, e.g. 'DATA' + 'BASE' = 'DATABASE'.
- Comparison: determine whether strings are equal or ordered.
- Substring: extract part of a string.
- Pattern matching: find a pattern inside text.
- Reversal: reverse character order.

Example: Reverse a String

```
INPUT s
reversed = empty string
FOR i from length(s)-1 down to 0
    append s[i] to reversed
PRINT reversed
```

Example: Input "CODE" produces "EDOC". A simple reversal takes O(n) time.

Pattern-Matching Algorithms

- Naive matching: tries the pattern at each possible position.
- KMP: uses prefix information to avoid repeated comparisons.
- Rabin–Karp: uses rolling hashes.
- Boyer–Moore: uses skip rules to reduce comparisons in many cases.

Applications include search boxes, text editors, spell-checkers, and DNA sequence analysis.

Chapter 5: Linked Lists

Definition

A **linked list** is a collection of nodes where each node stores data and link(s) to other node(s). Nodes need not occupy adjacent memory locations.

Types

Type	Definition	Example use
Singly linked list	Each node points to the next node.	Simple dynamic sequence
Doubly linked list	Each node links to previous and next nodes.	Forward/backward navigation
Circular singly linked list	Last node points to first node.	Round-robin processing
Circular doubly linked list	Previous and next links form a circle.	Circular navigation in both directions

Example

```
HEAD -> [10 | next] -> [20 | next] -> [30 | NULL]
```

To insert 15 after node 10, make the new node point to 20 and then update node 10's next link to the new node.

Typical Complexity

Operation	Typical time
Access/search by value	O(n)
Insert at head	O(1)
Delete head	O(1)
Insert after a known node	O(1)

Advantages: flexible size and efficient local insertion/deletion when a node reference is known. Disadvantages: no direct indexed access and extra memory for links.

Chapter 6: Stacks

Definition

A **stack** is a linear data structure that follows **LIFO (Last In, First Out)**: the most recently inserted item is removed first.

Example: A stack of plates: the top plate is usually removed first.

Operations

- Push: add an item to the top.
- Pop: remove and return the top item.
- Peek/Top: read the top item without removing it.
- Overflow: attempt to push onto a full fixed-capacity stack.
- Underflow: attempt to pop from an empty stack.

Example

```
PUSH(10)
PUSH(20)
PUSH(30)
POP() -> 30
TOP() -> 20
```

Complexity and Applications

Push, pop, and peek are typically O(1). Stacks are used in function calls, recursion, undo/redo, expression evaluation, bracket matching, and backtracking.

Chapter 7: Queues and Circular Queues

Definition

A **queue** is a linear data structure that follows **FIFO (First In, First Out)**: the earliest inserted item is removed first.

Operations and Types

- Enqueue: insert at the rear.
- Dequeue: remove from the front.
- Simple queue: standard FIFO behavior.
- Circular queue: the end wraps around to the beginning.
- Priority queue: removal follows priority.
- Deque: insertion and deletion can occur at both ends.

Example

```
ENQUEUE(10)
ENQUEUE(20)
ENQUEUE(30)
DEQUEUE() -> 10
FRONT() -> 20
```

Circular Queue

For an array of capacity N, a common next-index calculation is (index + 1) % N. This allows positions freed by dequeue operations to be reused.

Complexity and Uses

With a circular-array or linked implementation, enqueue and dequeue can be O(1). Queues are used in scheduling, printer jobs, request processing, and BFS.

Chapter 8: Recursion

Definition

Recursion is a technique in which a function calls itself on a smaller or simpler version of the problem.

Essential Parts

- Base case: stops further recursive calls.
- Recursive case: reduces the problem and calls the function again.

Example: Factorial

Factorial is defined as 0! = 1 and n! = n × (n–1)! for n > 0.

```
FACTORIAL(n)
    IF n <= 1
        RETURN 1
    RETURN n * FACTORIAL(n - 1)
```

Example: FACTORIAL(4) = 4 × 3 × 2 × 1 = 24.

Types, Advantages, and Limitations

- Direct recursion: a function calls itself.
- Indirect recursion: functions call each other in a cycle.
- Recursion can make tree algorithms and divide-and-conquer code clearer.
- Deep recursion uses call-stack memory and may cause stack overflow.

The factorial algorithm takes O(n) time and O(n) call-stack space.

Chapter 9: Searching Algorithms

Linear Search

Definition: Linear search checks elements one by one until the target is found or the collection ends.

Example: In [8, 3, 6, 9], searching for 6 checks 8, then 3, then 6.

Best time: O(1). Worst time: O(n). Iterative extra space: O(1).

Binary Search

Definition: Binary search repeatedly halves the search interval. The collection must be sorted, and the implementation must maintain a valid search range.

```
BINARY_SEARCH(A, target)
    low = 0
    high = length(A) - 1
    WHILE low <= high
        mid = low + (high - low) // 2
        IF A[mid] == target
            RETURN mid
        ELSE IF A[mid] < target
            low = mid + 1
        ELSE
            high = mid - 1
    RETURN -1
```

Example: For sorted A = [10, 20, 30, 40, 50], binary search finds 40 by repeatedly narrowing the range.

Worst time: O(log n). Iterative extra space: O(1).

Comparison

Feature	Linear Search	Binary Search
Requires sorted data	No	Yes
Worst-case time	O(n)	O(log n)
Suitable for	Small or unsorted collections	Sorted collections with efficient random access

Chapter 10: Sorting Algorithms

Definition

Sorting arranges elements in a chosen order, such as ascending or descending.

Main Algorithms

Algorithm	Idea	Best	Average	Worst
Bubble sort	Swap adjacent out-of-order elements.	O(n)*	$O(n^2)$	$O(n^2)$
Selection sort	Repeatedly select the minimum remaining element.	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion sort	Insert each item into a sorted prefix.	O(n)	$O(n^2)$	$O(n^2)$
Merge sort	Divide, sort, and merge.	O(n log n)	O(n log n)	O(n log n)
Quick sort	Partition around a pivot, then sort partitions.	O(n log n)	O(n log n)	$O(n^2)$
Heap sort	Build a heap and repeatedly extract the top item.	O(n log n)	O(n log n)	O(n log n)

*Bubble sort achieves O(n) best-case time only with an early-stop optimization.

Example: One Bubble-Sort Pass

```
Input:  [5, 2, 4, 1]
Compare 5 and 2 -> [2, 5, 4, 1]
Compare 5 and 4 -> [2, 4, 5, 1]
Compare 5 and 1 -> [2, 4, 1, 5]
```

After one pass, the largest element has moved to the final position. Further passes finish the sorting.

Stability and Space

- Merge sort is stable in its standard merge implementation and usually needs O(n) auxiliary space for arrays.
- Insertion sort is stable in its standard implementation and works well for small or nearly sorted inputs.
- Quick sort is often fast in practice, but its worst case is $O(n^2)$.
- Heap sort guarantees O(n log n) time and can be implemented in-place, but is not stable in its standard form.

Chapter 11: Trees and Binary Trees

Definition

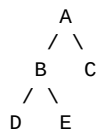
A **tree** is a non-linear structure made of nodes connected by edges. It represents hierarchical relationships.

Terminology

Term	Definition
Root	The topmost node
Parent / child	A node directly above / below another node
Leaf	A node with no children
Edge	A connection between two nodes
Depth	Number of edges from the root to a node
Height	Maximum number of edges on a downward path from a node to a leaf

Binary Tree

A **binary tree** is a tree in which each node has at most two children, called the left child and right child.



Properties

- With root at level 0, a binary tree has at most 2^L nodes at level L.
- A binary tree of height h has at most 2^(h+1) – 1 nodes.
- A non-empty tree with n nodes has n – 1 edges.

Common Types

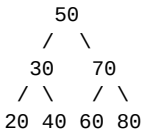
- Full: every node has either zero or two children.
- Complete: all levels except possibly the last are full; the last is filled left to right.
- Perfect: all internal nodes have two children and all leaves are at the same depth.
- Balanced: height is kept within a suitable bound to support efficient operations.

Chapter 12: Binary Search Trees

Definition

A **Binary Search Tree (BST)** is a binary tree ordered so that values in the left subtree are smaller than the node and values in the right subtree are greater, assuming distinct keys.

Example



Searching for 60: compare with 50, move right to 70, then left to 60.

Operations

- Search: compare target and move left or right.
- Insert: follow the ordering rule until an empty position is found.
- Delete leaf: remove it directly.
- Delete node with one child: connect its parent to its child.
- Delete node with two children: replace it with an inorder successor or predecessor, then remove that replacement node.

Complexity

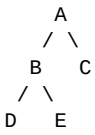
Operation	Balanced/average shape	Worst case
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)

A skewed BST may behave like a linked list. Self-balancing trees help maintain logarithmic height.

Chapter 13: Tree Traversals

Definition

A **tree traversal** is a systematic method for visiting every node in a tree.



Traversal	Rule	Output for the example
Inorder	Left, Root, Right	D, B, E, A, C
Preorder	Root, Left, Right	A, B, D, E, C
Postorder	Left, Right, Root	D, E, B, C, A
Level order	Visit level by level	A, B, C, D, E

Applications

- Inorder traversal of a BST visits keys in sorted order.

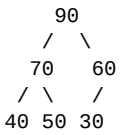
- Preorder is useful when serializing or copying tree structure.
- Postorder is useful when deleting a tree or evaluating some expression trees.
- Level order uses a queue and is useful for level-by-level processing.

Chapter 14: Heaps and Priority Queues

Definition

A **heap** is a complete binary tree that satisfies a heap-order property. In a max heap, each parent is at least as large as its children; in a min heap, each parent is at most as large as its children.

Example: Max Heap



Operations

Operation	Typical time
Find max in max heap / min in min heap	O(1)
Insert	O(log n)
Extract top priority	O(log n)
Build heap from an array	O(n)

Priority Queue

A **priority queue** removes the element with the highest or lowest priority rather than strictly following arrival order.

Example: In a hospital triage system, a critical case may be handled before a less urgent case.

Applications: scheduling, Dijkstra's algorithm, event simulation, and heap sort.

Chapter 15: Hashing and Hash Tables

Definition

Hashing maps a key to an index in a hash table using a hash function.

Example

```
h(key) = key % 10
h(27) = 7
h(37) = 7
```

Keys 27 and 37 collide because both map to index 7.

Collision Resolution

- Separate chaining: keep colliding entries in a list or collection at a bucket.
- Linear probing: check the next available slot sequentially.
- Quadratic probing: use quadratic increments to find another slot.
- Double hashing: use a second hash function to determine the probe step.

Complexity

Hash-table search, insertion, and deletion are often O(1) on average under suitable assumptions. Their worst-case time can be O(n), for example when many keys collide.

Applications

- Dictionaries and sets
- Caches
- Database lookup
- Compiler symbol tables
- Duplicate detection

Chapter 16: Graphs

Definition

A **graph** is a structure consisting of vertices (nodes) and edges (connections). It is often written as G = (V, E), where V is the set of vertices and E is the set of edges.

Types

- Directed graph: edges have directions.
- Undirected graph: edges have no direction.
- Weighted graph: edges carry weights or costs.
- Connected graph: every pair of vertices has a path between them.
- Cyclic graph: contains a cycle.
- Acyclic graph: contains no cycles.

Representations

Representation	Definition	Space
Adjacency matrix	A V × V matrix records edge relationships.	$O(V^2)$
Adjacency list	Each vertex stores its neighbors.	O(V + E)

Example: A road network can be modeled as a weighted graph: cities are vertices, roads are edges, and distances are weights.

Applications: maps, social networks, communication networks, dependency graphs, and web links.

Chapter 17: Breadth-First Search and Depth-First Search

BFS Definition

Breadth-First Search (BFS) explores a graph level by level from a starting vertex. It commonly uses a queue.

DFS Definition

Depth-First Search (DFS) explores as deeply as possible before backtracking. It uses recursion or an explicit stack.

Pseudocode

```
BFS(start):
    mark start visited
    enqueue(start)
    while queue not empty:
        v = dequeue()
        process(v)
        for each unvisited neighbor u of v:
            mark u visited
            enqueue(u)

DFS(v):
    mark v visited
    process(v)
    for each unvisited neighbor u of v:
        DFS(u)
```

Example

Example: For edges A-B, A-C, B-D, C-D, BFS from A may visit A, B, C, D. DFS may visit A, B, D, C. The order depends on neighbor ordering.

Comparison

Feature	BFS	DFS
Main structure	Queue	Stack / recursion
Exploration	Level by level	Deep path first
Shortest path	Finds shortest edge-count path in an unweighted graph	Does not generally guarantee shortest paths
Time with adjacency list	O(V + E)	O(V + E)

Chapter 18: Greedy Algorithms

Definition

A **greedy algorithm** makes the best-looking feasible choice at each step. It produces an optimal solution only for problems whose structure supports those choices.

Examples

- Activity selection: choose compatible activities, commonly in increasing finishing-time order.
- Fractional knapsack: select items by value-to-weight ratio when fractions are allowed.
- Kruskal's algorithm: add lowest-weight edges that do not create a cycle.
- Prim's algorithm: grow a minimum spanning tree by adding a cheapest connecting edge.
- Dijkstra's algorithm: find shortest paths when edge weights are non-negative.

Example: Fractional Knapsack

Example: If item A has value 60 and weight 10, its ratio is 6. If item B has value 100 and weight 20, its ratio is 5. When fractions are allowed, A is considered first because it gives greater value per unit weight.

Limitation

A locally optimal choice is not always globally optimal. For example, greedy selection by value-to-weight ratio does not always solve the 0/1 knapsack problem optimally.

Chapter 19: Divide and Conquer

Definition

Divide and Conquer solves a problem by dividing it into smaller subproblems, solving those subproblems, and combining their results.

Three Steps

- Divide: split the problem.
- Conquer: solve the smaller problems, often recursively.
- Combine: assemble the subproblem results.

Examples

- Binary search: discard half of the remaining search interval.
- Merge sort: sort halves and merge them.
- Quick sort: partition around a pivot and sort the partitions.

Example: Merge Sort

```
[8, 3, 6, 2]
Split -> [8, 3] and [6, 2]
Sort  -> [3, 8] and [2, 6]
Merge -> [2, 3, 6, 8]
```

Merge sort runs in O(n log n) time and typically uses O(n) auxiliary space for arrays.

Chapter 20: Dynamic Programming

Definition

Dynamic Programming (DP) solves a problem by storing and reusing answers to overlapping subproblems.

When DP Helps

- Overlapping subproblems: the same subproblems occur repeatedly.
- Optimal substructure: an optimal solution can be built from optimal solutions to subproblems.

Two Approaches

- Memoization: top-down recursion plus a cache.
- Tabulation: bottom-up table filling.

Example: Fibonacci

```
F[0] = 0
F[1] = 1
FOR i = 2 TO n
    F[i] = F[i-1] + F[i-2]
RETURN F[n]
```

Example: The first values are 0, 1, 1, 2, 3, 5, 8. This tabulation takes O(n) time. Storing the full table uses O(n) space; keeping only the last two values reduces auxiliary space to O(1).

Important DP Problems

- 0/1 Knapsack
- Longest Common Subsequence
- Longest Increasing Subsequence
- Coin Change

- Edit Distance
- Matrix Chain Multiplication

Greedy vs DP

Greedy commits to a local choice; DP systematically compares relevant subproblem solutions. DP may use more time and memory, but can solve problems for which a greedy strategy is not optimal.

Chapter 21: Backtracking

Definition

Backtracking builds a candidate solution incrementally and abandons a partial candidate when it cannot lead to a valid solution.

General Process

- Choose a possible next step.
- Check constraints.
- Continue if valid.
- Undo the choice if it fails.
- Try another option.

Examples

- N-Queens: place queens so that no two attack each other.
- Sudoku: fill cells while satisfying row, column, and box constraints.
- Rat in a maze: find a path through valid cells.
- Graph coloring: color adjacent vertices differently.

Example: Maze

Example: If a path reaches a blocked cell, the algorithm returns to the previous decision point and tries another available direction.

Backtracking can prune many invalid candidates, but worst-case time is often exponential.

Chapter 22: Time and Space Complexity

Time Complexity

Time complexity describes how the number of basic operations grows as input size n increases.

Space Complexity

Space complexity describes how memory usage grows with input size. Auxiliary space refers to extra working memory, excluding the input when that distinction is made.

Examples

```
Single loop:
FOR i = 1 TO n
    PRINT i
Time: O(n)

Two nested loops:
FOR i = 1 TO n
    FOR j = 1 TO n
        PRINT i, j
Time: O(n²)
```

Simplification Rules

- Ignore constant factors: $O(3n) = O(n)$.
- Keep the dominant term: $O(n^2 + n + 5) = O(n^2)$.
- Sequential sections add; the largest growth term dominates.
- For nested loops, count how many times the inner operation runs.
- For recursive algorithms, use recurrence relations where appropriate.

Chapter 23: Advanced Topics

AVL Trees

An **AVL tree** is a self-balancing Binary Search Tree. At every node, the heights of the left and right subtrees differ by at most one. Rotations restore balance after updates.

- LL case: right rotation.
- RR case: left rotation.
- LR case: left rotation on child, then right rotation.
- RL case: right rotation on child, then left rotation.

Search, insertion, and deletion take O(log n) time in an AVL tree.

Trie

A **Trie** is a tree that stores strings by sharing common prefixes.

Example: The words 'car', 'cat', and 'can' share the prefix 'ca'.

Applications: autocomplete, dictionaries, prefix search, and spell-checking.

Disjoint Set Union (Union-Find)

A **Disjoint Set Union (DSU)** structure maintains disjoint groups. Find returns a set representative; Union merges two sets. Path compression and union by rank/size make operations very efficient.

Application: detecting connectivity and supporting Kruskal's minimum spanning tree algorithm.

File Structures

File organization defines how records are stored and retrieved. Common approaches include sequential, direct/random, indexed, and indexed-sequential access.

Minimum Spanning Tree

A **minimum spanning tree (MST)** of a connected, weighted, undirected graph connects all vertices with no cycles and minimum total edge weight. Prim's and Kruskal's algorithms are standard methods.

Chapter 24: Exam Questions and Quick Revision

Short-Answer Questions

- Define data structure and give two examples.
- Define an algorithm and list its characteristics.
- What do O, Ω, and Θ mean?
- What is the difference between an array and a linked list?
- Explain LIFO and FIFO.
- What are recursion and a base case?

- State the prerequisite for binary search.
- Define a Binary Search Tree.
- What are inorder, preorder, and postorder traversals?
- What is a heap?
- Define hashing and collision.
- Differentiate BFS and DFS.
- What is a greedy algorithm?
- What are memoization and tabulation?
- Define backtracking and minimum spanning tree.

Long-Answer Questions

- Explain the classification of data structures with examples.
- Explain array insertion and deletion.
- Describe singly, doubly, and circular linked lists.
- Explain stack and queue operations with examples.
- Compare linear search and binary search.
- Explain Bubble Sort, Insertion Sort, Merge Sort, and Quick Sort.
- Explain binary tree properties and traversal methods.
- Explain BST insertion, search, and deletion.
- Describe heap operations and hashing collision resolution.
- Explain graph representations, BFS, and DFS.
- Explain Prim's and Kruskal's algorithms.
- Explain Dynamic Programming with Fibonacci or Knapsack.
- Explain Backtracking using N-Queens.

Practice Programs

- Find the maximum and minimum in an array.
- Reverse an array or string.
- Implement linear and binary search.
- Implement Bubble Sort and Insertion Sort.
- Implement Merge Sort and Quick Sort.
- Create a singly linked list with insertion and deletion.
- Implement stack and queue operations.
- Calculate factorial using recursion.
- Perform tree traversals.
- Implement BST search and insertion.
- Implement BFS and DFS.
- Solve Fibonacci using Dynamic Programming.
- Demonstrate hashing with collision handling.

Complexity Cheat Sheet

Structure / Algorithm	Key complexity
Array indexed access	O(1)
Linear search	O(n)
Binary search	O(log n)
Linked-list search	O(n)
Stack push/pop	O(1)
Queue enqueue/dequeue	O(1) with suitable implementation
BST search	O(log n) average for suitable shapes; O(n) worst
Heap insertion / extract-top	O(log n)
Hash-table lookup	O(1) expected; O(n) worst
BFS / DFS with adjacency lists	O(V + E)
Merge sort	O(n log n)
Quick sort	O(n log n) average; $O(n^2)$ worst
Heap sort	O(n log n)
Fibonacci tabulation	O(n) time

Final Study Tip

For every topic, learn (1) the definition, (2) a small worked example, (3) the algorithm or diagram, (4) time and space complexity, and (5) one practical application. Practice writing algorithms without looking at the notes.